Mathematical Background of the Duotrigordle Solver

Abstract

This note describes the model and the decision rule implemented in the files `duotrigordle_solver.py` and `duo_bridge_server.py`. The solver keeps an exact set of consistent candidates per board, puts a heuristic prior on each set, and chooses every guess greedily by minimising the expected logarithmic size of the candidate sets after the guess. Already determined boards are cleared first.

Contents

1 Game model **1**
1.1 Feedback function 2

2 State and candidate sets **2**
2.1 Fallback for missing targets 2

3 Prior distribution per board **3**

4 Scoring a guess **3**
4.1 Per-board feedback distribution 3
4.2 Relation between ELog and information gain 4
4.3 Candidate features 4

5 Decision rule **5**

6 Bounds on the number of guesses **5**

7 Computational cost **5**

8 Limitations **6**

1 Game model

Let Σ be the alphabet and $L = 5$ the word length. We use three word lists:

- $\mathcal{A} \subseteq \Sigma^L$, the solution list (`solutions_en.txt`, $|\mathcal{A}| = 2313$),
- $\mathcal{G} \subseteq \Sigma^L$, the allowed guesses (`guesses_en.txt`, $|\mathcal{G}| = 14855$),
- $\mathcal{V} = \mathcal{G} \cup \mathcal{A}$, all valid words.

A game consists of $M = 32$ boards with pairwise distinct target words $t_1, \ldots, t_M \in \mathcal{V}$ and at most $K = 37$ guesses. Every guess g_k is applied to all boards that are not yet solved.

1.1 Feedback function

Definition 1 (Wordle feedback)**.** For a guess g and a target t, the feedback $f(g,t) \in \{0,1,2\}^L$ (grey, yellow, green) is computed in two passes. First,

$$f_i(g,t) = 2 \quad \text{if } g_i = t_i.$$

Let $r(c) = \#\{\, j : t_j = c,\ g_j \neq t_j \,\}$ be the number of letters c in t that are not used up by green positions. Then, going through the non-green positions $i = 1, \ldots, L$ from left to right,

$$f_i(g,t) = \begin{cases} 1 & \text{if } r(g_i) > 0, \text{ and afterwards } r(g_i) \leftarrow r(g_i) - 1, \\ 0 & \text{otherwise.} \end{cases}$$

The definition handles repeated letters exactly. For example, $f(\texttt{eerie}, \texttt{there}) = (1,0,1,0,2)$: the target has two `e`s, one is used by the green last position, so only the first of the remaining guessed `e`s turns yellow.

Feedback is encoded as an integer $\kappa(\varphi) = \sum_{i=1}^{L} \varphi_i \, 3^{\,i-1} \in \{0, \ldots, 242\}$. The all-green pattern $\mathbf{2} = (2,2,2,2,2)$ has the code $\kappa(\mathbf{2}) = 242$.

Lemma 1 (Vectorised yellow rule)**.** *For a non-green position i let*

$$a_i = \#\{\, j : t_j = g_i \,\} - \#\{\, j : t_j = g_i,\ g_j = t_j \,\}, \qquad u_i = \#\{\, j < i : f_j = 1,\ g_j = g_i \,\}.$$

Then $f_i(g,t) = 1 \iff u_i < a_i$.

Proof. a_i equals the initial value of $r(g_i)$, and u_i counts how often $r(g_i)$ has already been decremented before position i. So $r(g_i) > 0$ at step i exactly when $a_i - u_i > 0$. □

Lemma 1 lets `numpy_feedback_codes` and `cupy_feedback_codes` compute a full matrix of feedback codes for B guesses against N targets with array operations of size $B \times N \times L$.

2 State and candidate sets

After k guesses with observed feedback $\varphi_{b,1}, \ldots, \varphi_{b,k}$ on board b, the candidate set is

$$C_b^{(k)} = \{\, t \in C_b^{(0)} : f(g_j, t) = \varphi_{b,j} \text{ for all } j \leq k \,\}, \qquad C_b^{(0)} = \mathcal{A}.$$

It can be updated incrementally: $C_b^{(k)} = \{t \in C_b^{(k-1)} : f(g_k, t) = \varphi_{b,k}\}$. Board b is *solved* at step $\tau_b = k$ if $\varphi_{b,k} = \mathbf{2}$. The set of active boards is $\mathcal{B} = \{b : b \text{ not solved}\}$. A board with $|C_b| = 1$ is called a *singleton*: its target is known, but it still costs one guess to enter it.

2.1 Fallback for missing targets

The live website may use a solution list different from $\mathcal{A}$. If the update yields $C_b^{(k)} = \emptyset$, the server rebuilds this board alone from all valid words:

$$C_b^{(k)} = \{\, t \in \mathcal{V} : f(g_j, t) = \varphi_{b,j} \text{ for all } j \leq k \,\}.$$

Every target must be typeable, hence $t_b \in \mathcal{V}$ and the rebuilt set always contains t_b. If it is still empty, the observed feedback itself must be wrong (for example a misread colour). The other boards keep the smaller list $\mathcal{A}$, so the cost of the fallback is paid only where it is needed.

3 Prior distribution per board

The candidates of a board are not treated as equally likely. For each active board, the function `compute_board_model` computes local statistics over C_b:

$$\ell_b(c) = \frac{\#\{\, w \in C_b : c \in w \,\}}{|C_b|} \qquad \text{(share of candidates containing letter } c\text{)},$$

$$\pi_b(i,c) = \frac{\#\{\, w \in C_b : w_i = c \,\}}{|C_b|} \qquad \text{(share with letter } c \text{ at position } i\text{)}.$$

Writing $U(w)$ for the set of distinct letters of w, each candidate $w \in C_b$ gets four features in $[0,1]$:

$$x_1(w) = \frac{\log(1+n(w))}{\log(1+\max_v n(v))} \qquad \text{(frequency in the source list)},$$

$$x_2(w) = \frac{1}{|U(w)|} \sum_{c \in U(w)} \ell_b(c) \qquad \text{(mean letter coverage)},$$

$$x_3(w) = \frac{1}{L} \sum_{i=1}^{L} \pi_b(i, w_i) \qquad \text{(mean positional frequency)},$$

$$x_4(w) = \frac{z(w)}{\max_v z(v)} \qquad \text{(Zipf frequency from } \texttt{wordfreq}\text{)}.$$

With normalised weights $\alpha_j \geq 0$, $\sum_j \alpha_j = 1$ (`WEIGHTS`, default $0.2, 0.3, 0.3, 0.2$; $\alpha_4 = 0$ if `wordfreq` is not installed) the prior is

$$p_b(w) = \frac{s_b(w)}{\sum_{v \in C_b} s_b(v)}, \qquad s_b(w) = \max\Big(0,\ \sum_{j=1}^{4} \alpha_j\, x_j(w)\Big),$$

and $p_b(w) = 0$ for $w \notin C_b$. If all scores are zero, the prior is uniform.

Remark. Each word appears exactly once in the shipped lists, so x_1 is constant and does not change the ranking. The prior is a heuristic, not a calibrated probability. Its main job is to give common English words more weight than obscure ones.

4 Scoring a guess

4.1 Per-board feedback distribution

Fix an active board b with $|C_b| \geq 2$ and a guess $g \in \mathcal{G}$. The guess partitions C_b into classes

$$C_b^{\varphi}(g) = \{\, t \in C_b : f(g,t) = \varphi \,\}, \qquad n_\varphi = |C_b^{\varphi}(g)|, \qquad q_\varphi = \sum_{t \in C_b^{\varphi}(g)} p_b(t).$$

Here q_φ is the prior probability of observing φ. From this partition we compute three quantities (`pattern_metrics`):

$$H_b(g) = -\sum_{\varphi} q_\varphi \log_2 q_\varphi \qquad \text{(InfoGain: entropy of the feedback)},$$

$$R_b(g) = \sum_{\varphi \neq \mathbf{2}} q_\varphi\, n_\varphi \qquad \text{(ERest: expected number of remaining candidates)},$$

$$E_b(g) = \sum_{\varphi \neq \mathbf{2},\ n_\varphi > 0} q_\varphi \log_2 n_\varphi \qquad \text{(ELog: expected remaining uncertainty in bits)}.$$

The green class is excluded from R_b and E_b because the board is solved in that case. Singletons ($|C_b| = 1$) contribute $H_b = R_b = E_b = 0$: a wrong guess cannot teach anything about a board whose target is already known.

The global scores are sums over the active boards:

$$H(g) = \sum_{b \in \mathcal{B}} H_b(g), \qquad R(g) = \sum_{b \in \mathcal{B}} R_b(g), \qquad E(g) = \sum_{b \in \mathcal{B}} E_b(g).$$

4.2 Relation between ELog and information gain

Proposition 1. *If the prior on board b is uniform, $p_b(t) = 1/|C_b|$, then*

$$E_b(g) = \log_2 |C_b| - H_b(g).$$

Hence, for uniform priors on all boards, minimising $E(g)$ is equivalent to maximising $H(g)$.

Proof. With a uniform prior, $q_\varphi = n_\varphi / |C_b|$, so

$$H_b(g) = -\sum_{\varphi} q_\varphi \log_2 \frac{n_\varphi}{|C_b|} = \log_2 |C_b| - \sum_{\varphi} q_\varphi \log_2 n_\varphi.$$

The green class has $n_{\mathbf{2}} \leq 1$ and therefore contributes $\log_2 1 = 0$ to the last sum, so leaving it out does not change the value. The last sum is exactly $E_b(g)$. The term $\log_2 |C_b|$ does not depend on g. □

With a non-uniform prior the two criteria differ: H_b measures how unpredictable the feedback is, whereas E_b measures how many candidates are expected to remain, in $\log_2$ scale and weighted by the prior. The solver uses E as its main criterion because it is directly tied to the remaining work: it is the expected number of bits still needed to pin down the targets, summed over all boards.

4.3 Candidate features

In addition to the information measures, every guess gets direct-solve features:

$$\begin{aligned}
N(g) &= \#\{\, b \in \mathcal{B} : g \in C_b \,\} && \text{(boards where } g \text{ is a candidate)},\\
S(g) &= \#\{\, b \in \mathcal{B} : C_b = \{g\} \,\} && \text{(singleton boards solved by } g),\\
P_\Sigma(g) &= \sum_{b \in \mathcal{B}} p_b(g) && \text{(expected number of boards solved directly)},\\
P_\exists(g) &= 1 - \prod_{b \in \mathcal{B}} (1 - p_b(g)) && \text{(probability to solve at least one board)},\\
\Lambda(g) &= \sum_{b \in \mathcal{B}} \sum_{c \in U(g)} \ell_b(c) && \text{(letter coverage)},\\
\Pi(g) &= \sum_{b \in \mathcal{B}} \sum_{i=1}^{L} \pi_b(i, g_i) && \text{(position coverage)}.
\end{aligned}$$

P_Σ follows from linearity of expectation. $P_\exists$ treats the boards as independent, which is an approximation (see Section 8). A guess with $N(g) > 0$ is a *candidate guess*, otherwise a *probe guess*.

Remark. Because the targets are pairwise distinct, two boards cannot both have the singleton set $\{g\}$ if the feedback is consistent. In practice $S(g) \in \{0, 1\}$.

5 Decision rule

Let lex min denote the minimum under lexicographic order of tuples. The rule implemented in `choose_next_guess` is:

Algorithm 1 Choice of the next guess

Require: active boards $\mathcal{B}$, candidate sets C_b, scores for all $g \in \mathcal{G}$

if some $b \in \mathcal{B}$ has $|C_b| = 1$ **and** $\mathcal{G}_S = \{g : S(g) > 0\} \neq \emptyset$ **then**
  return $\operatorname{lex\,min}_{g \in \mathcal{G}_S}(-S(g),\ E(g),\ -P_\Sigma(g),\ -N(g),\ -H(g),\ -\Lambda(g),\ -\Pi(g))$
end if
$g^\star \leftarrow \operatorname{lex\,min}_{g \in \mathcal{G}}(E(g),\ -P_\Sigma(g),\ -N(g),\ -P_\exists(g),\ -H(g),\ -\Lambda(g),\ -\Pi(g),\ -x_4(g))$
if $g^\star$ is a probe **and** margin $m > 0$ **then**
  $g^c \leftarrow$ best candidate guess under the same order
  if $E(g^c) \leq E(g^\star) + m$ **then**
    return g^c
  end if
end if
return $g^\star$

The singleton rule reflects that a known board must be entered at some point anyway. Among the singleton words, the one that also splits the remaining boards best is chosen. The margin m (`ATTEMPT_REST_MARGIN`, default 0) can prefer a guess that might solve a board over a slightly more informative probe.

6 Bounds on the number of guesses

Proposition 2. *Every game needs at least M guesses, and the mean solve index satisfies*

$$\bar{\tau} = \frac{1}{M}\sum_{b=1}^{M} \tau_b \ \geq\ \frac{M+1}{2} = 16.5.$$

Proof. A guess g solves board b only if $g = t_b$. Since the targets are pairwise distinct, each guess solves at most one board, so at least M guesses are needed. The solve indices τ_b are therefore pairwise distinct positive integers, and their sum is at least $1 + 2 + \cdots + M = M(M+1)/2$. □

Every probe guess and every candidate guess that solves no board pushes all later solve indices back by one. The first few guesses are almost always of this kind, because no board can be identified before some information is collected. The score $\bar{\tau}$ therefore measures how quickly the solver moves from exploration to clearing boards.

7 Computational cost

One decision evaluates the feedback of every guess against every candidate on every active non-singleton board:

$$\text{pairs per turn} = |\mathcal{G}| \cdot \sum_{b \in \mathcal{B},\ |C_b| \geq 2} |C_b|.$$

For the first guess this gives $14855 \cdot 32 \cdot 2313 \approx 1.10 \cdot 10^9$ pairs, and $14855 \cdot 32 \cdot 14855 \approx 7.06 \cdot 10^9$ pairs with `--wide-solutions`. These numbers match the progress output of the server. The first decision is identical in every game, so it is cached. The candidate sets shrink quickly. In the example run with `--wide-solutions` (`examples/daily-2026-11-03.log`) the median set size was 287 after one guess and 11 after two.

8 Limitations

- **Greedy.** The rule looks one guess ahead. It does not plan sequences of guesses, so it is not guaranteed to be optimal.
- **Independent boards.** Each board is modelled separately. The coupling through distinct targets ($t_b \neq t_{b'}$) is ignored; when one board is solved with w, w is not removed from the other boards' candidate sets. $P_\exists$ also assumes independence.
- **Heuristic prior.** The weights α_j were chosen by hand and are not fitted to real target distributions. Fitting them to the logs in `output/live_games.jsonl` is a natural next step.
- **Unknown solution list.** The prior only covers $\mathcal{A}$ until a board is widened. In widened boards, x_4 (Zipf frequency) is the only feature that separates common words from rare ones; x_2 and x_3 only describe letter patterns.